

ISM 6251 and 4543: Machine Learning (Fall 2025)

Assignment 1

- Please provide succinct answers to the questions below.
- You should submit a .ipynb file and a pdf (for any non-coding questions) in Canvas.
- Please include the names of all team-members in your write up and in the name of the file. One submission per group.
- You should include all the Python functions you use.

Problem 1. Download the chickwts.csv from Canvas and do the following tasks.

- (a) How many instances and variables are in this data set? How do you get this information without viewing data?

Two variables (weight and feed) and 71 instances. This is observable through the info() function:

```
# Problem 1. (a)
# How many instances and variables are in this data set?
# How do you get this information without viewing data?

print(chick.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 71 entries, 0 to 70
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0    weight    71 non-null      int64
1    feed      71 non-null      object
dtypes: int64(1), object(1)
memory usage: 1.2+ KB
None
```

- (b) The column “weight” means weights in grams. Add a “new” column that gives the weight in kilograms, rather than grams. Make sure the column has a descriptive name.

```
# Problem 1. (b)
# (b) The column “weight” means weights in grams.
# Add a “new” column that gives the weight in kilograms, rather than grams.
# Make sure the column has a descriptive name.
chick["weightkg"] = chick.weight / 1000
print(chick.head())
```

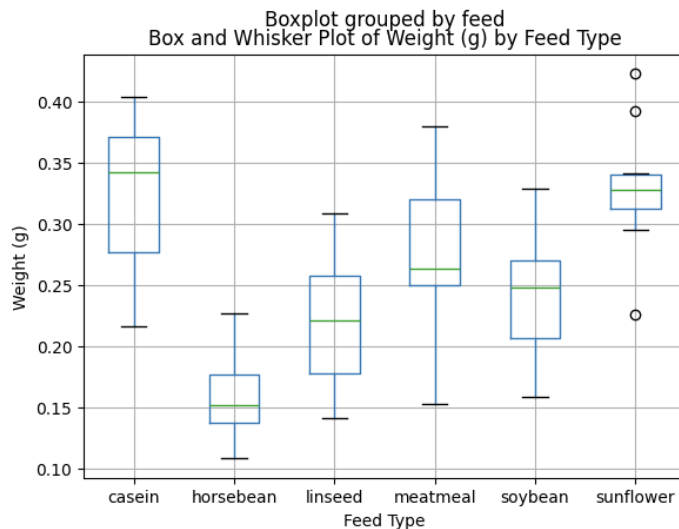
```
   weight    feed  weightkg
0    179  horsebean    0.179
1    160  horsebean    0.160
2    136  horsebean    0.136
3    227  horsebean    0.227
4    217  horsebean    0.217
```

- (c) Make a box-and-whiskers plot showing the distribution of chicken weight according to feed type. Make sure to label the axes appropriately.

```
# Problem 1. (c)
# Make a box-and-whiskers plot showing the distribution of chicken weight according to feed type.
# Make sure to label the axes appropriately.

import matplotlib.pyplot as plt

chick.boxplot(column="weightkg", by="feed")
plt.title("Box and Whisker Plot of Weight (g) by Feed Type")
plt.xlabel("Feed Type")
plt.ylabel("Weight (g)")
plt.show()
```



Problem 2. Download the OVCclinicalinfo.csv from Canvas. Import the spreadsheet and name it as a dataframe “clin”. Briefly inspect the data.
(a) Summarize the distribution of each variable in this data set.

```
# Problem 2. (a)
# Summarize the distribution of each variable in this data set.
```

```
# Numerical
print(clin.describe())
```

```
# Categorical
print(clin.describe(include='object'))
```

	Survival	0 = alive	1 = dead	Assigned Stage	response 0=NR, 1=CR
count	119.000000		119.000000	118.000000	119.000000
mean	49.268908		0.579832	3.152542	0.714286
std	41.403492		0.495673	0.384021	0.453664
min	1.000000		0.000000	2.000000	0.000000
25%	16.000000		0.000000	3.000000	0.000000
50%	34.000000		1.000000	3.000000	1.000000
75%	74.000000		1.000000	3.000000	1.000000
max	185.000000		1.000000	4.000000	1.000000

	OVC TumorID	GRADE	Debulk	CA125.POST
count	119	118	119	114
unique	119	7	5	86
top	0.08	3	5	7
freq	1	55	49	7

- (b) Remove the variable “Survival” from this data set. Convert the first column to character type. Change the name of the third column to “event”. Convert the “CA125.POST” and “GRADE” columns into numeric values, with the ambiguous entries coded as NAs. Fix the “Debulk” column such that only two levels are used (“O” and “S”). Rename the eighth column to “response” and convert it to a binary variable (0 = FALSE, 1 = TRUE)

```
# Problem 2. (b) - i
#Remove the variable “Survival” from this data set.
```

```
clin = clin.drop(columns=["Survival"])
```

```
clin.head()
```

	OVC TumorID	0 = alive 1 = dead	Assigned Stage	GRADE	Debulk	CA125.POST	response 0=NR, 1=CR
0	0.08	1	4.0	3	S	72.3	0
1	860	1	4.0	3	O	133	0
2	872	0	3.0	3	S	3.1	1
3	922	0	3.0	2	S	9	1
4	1024	1	4.0	3	S	12	1

```
# Problem 2. (b) - ii
# Convert the first column to character type.
```

```
first_col = clin.columns[0]
clin[first_col] = clin[first_col].astype("string")
```

```
print(clin[first_col].head())
```

```
0    0.08
1    860
2    872
3    922
4   1024
Name: OVC TumorID, dtype: string
```

```
# Problem 2. (b) - iii
# Change the name of the third column to "event".
```

```
third_col = clin.columns[2]
clin = clin.rename(columns= {third_col: "event"})
```

```
clin.head()
```

	OVC TumorID	0 = alive 1 = dead	event	GRADE	Debulk	CA125.POST	response 0=NR, 1=CR
0	0.08	1	4.0	3	S	72.3	0
1	860	1	4.0	3	O	133	0
2	872	0	3.0	3	S	3.1	1
3	922	0	3.0	2	S	9	1
4	1024	1	4.0	3	S	12	1

```
# Problem 2. (b) - iv
# Convert the "CA125.POST" and "GRADE" columns into numeric values, with the ambiguous entries coded as NAs.
```

```
clin["CA125.POST"] = pd.to_numeric(clin["CA125.POST"], errors="coerce")
clin["GRADE"] = pd.to_numeric(clin["GRADE"], errors="coerce")
```

```
clin.iloc[26]
```

	26
OVC TumorID	2064
0 = alive 1 = dead	1
event	3.0
GRADE	3.0
Debulk	O
CA125.POST	NaN
response 0=NR, 1=CR	0

```
dtype: object
```

```

# Problem 2. (b) - v
# Fix the "Debulk" column such that only two levels are used ("O" and "S").

deb = (
    clin["Debulk"]
    .astype("string")
    .str.strip()
    .str.upper()
    .str[0] # Argument to take first letter (handles "Optimal" and "Suboptimal" entries)
)
clin["Debulk"] = deb.where(deb.isin(["O", "S"]))

clin.head(100)

```

	OVC TumorID	0 = alive 1 = dead	event	GRADE	Debulk	CA125.POST	response 0=NR, 1=CR
0	0.08	1	4.0	3.0	S	72.3	0
1	860	1	4.0	3.0	O	133.0	0
2	872	0	3.0	3.0	S	3.1	1
3	922	0	3.0	2.0	S	9.0	1
4	1024	1	4.0	3.0	S	12.0	1
...	...	...	...	...	...	...	...
95	M1572	1	3.0	3.0	O	49.0	0
96	M17	0	4.0	2.0	S	4700.0	0
97	M1891	0	4.0	1.0	S	17.0	1
98	M2070	0	3.0	3.0	S	22.0	1
99	M2097	0	2.0	NaN	O	NaN	1

100 7 1

```
# Problem 2. (b) - vi
# Rename the eighth column to "response" and convert it to a binary variable (0 = FALSE, 1 = TRUE)

eighth_col = clin.columns[6]
clin = clin.rename(columns={eighth_col: "response"})

clin["response"] = (
    pd.to_numeric(clin["response"], errors="coerce")
    .where(lambda s: s.isin([0, 1]))
    .astype("UInt8")
)

clin.head(5)
```

	OVC TumorID	0 = alive 1 = dead	event	GRADE	Debulk	CA125.POST	response
0	0.08	1	4.0	3.0	S	72.3	0
1	860	1	4.0	3.0	O	133.0	0
2	872	0	3.0	3.0	S	3.1	1
3	922	0	3.0	2.0	S	9.0	1
4	1024	1	4.0	3.0	S	12.0	1

(c) Is there any missing values or outliers in this data set? If yes, how do you handle them? Justify your choice.

```
# Problem 2. (c)
# Is there any missing values or outliers in this data set?
print(clin.isna().sum())
```

```
OVC TumorID      0
0 = alive  1 = dead  0
event        1
GRADE        5
Debulk        0
CA125.POST    11
response      0
dtype: int64
```

Yes, as seen above, there are rows with missing data in columns [event], [GRADE] and [CA125.POST]. There are multiple techniques in handling situations like this, including simply dropping the entries with missing values in cases where they represent a very small percentage of the total number of rows. Another simple common approach is replacing the missing values with the attribute's median in case of numerical variables, or the mode, if categorical. Nevertheless, there is always a risk of compromising the understanding of relationships between variables in this approach, once only the missing value is being modified. A mitigation technique is creating a new Boolean column to track rows that originally had missing values by notating them with 0s and 1s. This would be my personal choice to deal with this small database that is small in size but do count with about 10% of rows with missing values. Other more advances techniques include the K-Nearest Neighbors (KNN) Imputation, which replaces missing values considering the values across multiple variables of the most similar rows to the one being manipulated.

Problem 3. Download the “hmeq.csv” data from Canvas. Write down a Python code that perform data cleaning such as dealing with missing values, and removes all the outliers with respect to all the numerical variables “together”.

Hint: You can write your own function (lets call it RemoveOutlier; see below) that receives the variable x as its input and returns the values of x that are not outlier. Be creative.

```
def RemoveOutlier(x)
    Your code
    return( object that you need to use outside of the function )
```

Before and After Comparison

Before

```
# Understanding the data before it is cleaned
# [BAD] is the target and a binary flag with no missing entries

data = pd.read_csv("hmeq.csv")

summary = pd.DataFrame({
    "dtype": data.dtypes.astype(str),
    "non_null": data.notna().sum(),
    "missing": data.isna().sum(),
    "missing_%": (100 * data.isna().mean()).round(2),
    "unique": data.nunique(dropna=True)
})

summary
```

	dtype	non_null	missing	missing_%	unique
BAD	int64	5960	0	0.00	2
LOAN	int64	5960	0	0.00	540
MORTDUE	float64	5442	518	8.69	5053
VALUE	float64	5848	112	1.88	5381
REASON	object	5708	252	4.23	2
JOB	object	5681	279	4.68	6
YOJ	float64	5445	515	8.64	99
DEROG	float64	5252	708	11.88	11
DELINQ	float64	5380	580	9.73	14
CLAGE	float64	5652	308	5.17	5314
NINQ	float64	5450	510	8.56	16
CLNO	float64	5738	222	3.72	62
DEBTINC	float64	4693	1267	21.26	4693

After

```
# Assessing the data before it is cleaned
summary = pd.DataFrame({
    "dtype": clean_drop.dtypes.astype(str),
    "non_null": clean_drop.notna().sum(),
    "missing": clean_drop.isna().sum(),
    "missing_%": (100 * clean_drop.isna().mean()).round(2),
    "unique": clean_drop.nunique(dropna=True)
})

summary
```

	dtype	non_null	missing	missing_%	unique
BAD	int64	5754	0	0.0	2
LOAN	int64	5754	0	0.0	479
MORTDUE	float64	5754	0	0.0	4857
VALUE	float64	5754	0	0.0	5185
REASON	object	5754	0	0.0	2
JOB	object	5754	0	0.0	6
YOJ	float64	5754	0	0.0	99
DEROG	float64	5754	0	0.0	11
DELINQ	float64	5754	0	0.0	14
CLAGE	float64	5754	0	0.0	5125
NINQ	float64	5754	0	0.0	16
CLNO	float64	5754	0	0.0	62
DEBTINC	float64	5754	0	0.0	4517

Function Created

```
# 1) Define column groups because only continuous data will be included in this moderate outlier analysis
count_like = [c for c in ["DEROG", "DELINQ", "NINQ", "CLNO", "BAD"] if c in data.columns]
num_cols = data.select_dtypes(include="number").columns.tolist()
cont_cols = [c for c in num_cols if c not in count_like] # continuous only

# 2) Outlier flag (IQR with looser fence k=3.0.)
# We were originally looking at the usual IQR rule of k=1.5 but that led to too many entries being dropped
def is_outlier_iqr(s: pd.Series, k: float = 3.0) -> pd.Series:
    s = pd.to_numeric(s, errors="coerce")
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    if pd.isna(iqr) or iqr == 0:
        return pd.Series(False, index=s.index)
    low, high = q1 - k*iqr, q3 + k*iqr
    return (s < low) | (s > high)

# 3) Drop rows that are outliers in any continuous column
flags = pd.DataFrame({c: is_outlier_iqr(data[c], k=3.0) for c in cont_cols})
keep_mask = ~flags.any(axis=1)
clean_drop = data[keep_mask].copy()
print(f"Dropped rows: {len(data) - len(clean_drop)} ")

# 4) Replace missing values
# We are using the median for numeric values
for c in cont_cols:
    clean_drop[c] = clean_drop[c].fillna(clean_drop[c].median())

# And the mode for count like values that display mostly 0s
for c in count_like:
    if c in clean_drop.columns:
        clean_drop[c] = clean_drop[c].fillna(0)

# For categoricals (Reason & Job) we are going with the mode
cat_cols = clean_drop.select_dtypes(exclude="number").columns
for c in cat_cols:
    m = clean_drop[c].mode(dropna=True)
    if not m.empty:
        clean_drop[c] = clean_drop[c].fillna(m.iloc[0])
```

Dropped rows: 206

Problem 4. Use the “hmeq.csv” to construct a decision tree model.

1. Before modeling, split the data into two subsets: training (70%) and test (30%).

```
# Split the data into two subsets: training (70%) and test (30%)

from sklearn.model_selection import train_test_split

# Separate features (X) and target (y)
X = clean_drop.drop(columns=["BAD"])
y = clean_drop["BAD"]

# Split into train (70%) and test (30%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print(len(X_train))
print(len(X_test))
```

```
4027
1727
```

2. Ensure that the target variable BAD is a flag variable.

```
# Ensure that the target variable BAD is a flag variable.
# This was already confirmed by the above summary statement in Problem 3,
# which returned that column BAD has no missing values and only 2 unique values.

valid = set(clean_drop["BAD"].unique()) <= {0, 1}
if valid:
    print("BAD is a binary flag.")
else:
    print("BAD contains values outside 0 and 1.")
```

```
BAD is a binary flag.
```

3. Then build the decision tree and check the prediction result using the default settings, and take a look at the tree by printing the rules and plotting the tree. Explain and discuss the rules and plot briefly.
The tree is highly detailed (depth = 22, 388 leaves), giving strong accuracy of 87%. Rules confirm that our tree prioritizes delinquencies, debt ratio, loan size, and property value in classification. The visualization highlights how quickly the tree fragments the dataset, suggesting pruning or limiting depth could improve generalization while still preserving interpretability.

```

# Default Decision tree

from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.metrics import accuracy_score, roc_auc_score, classification_report
import matplotlib.pyplot as plt
import pandas as pd

# Encoding categoricals into dummy variables
X_train_encoded = pd.get_dummies(X_train, columns=['REASON', 'JOB'], drop_first=True)
X_test_encoded = pd.get_dummies(X_test, columns=['REASON', 'JOB'], drop_first=True)

# Training the default tree (gini criterion)
dt_default = DecisionTreeClassifier(random_state=42)
dt_default.fit(X_train_encoded, y_train)

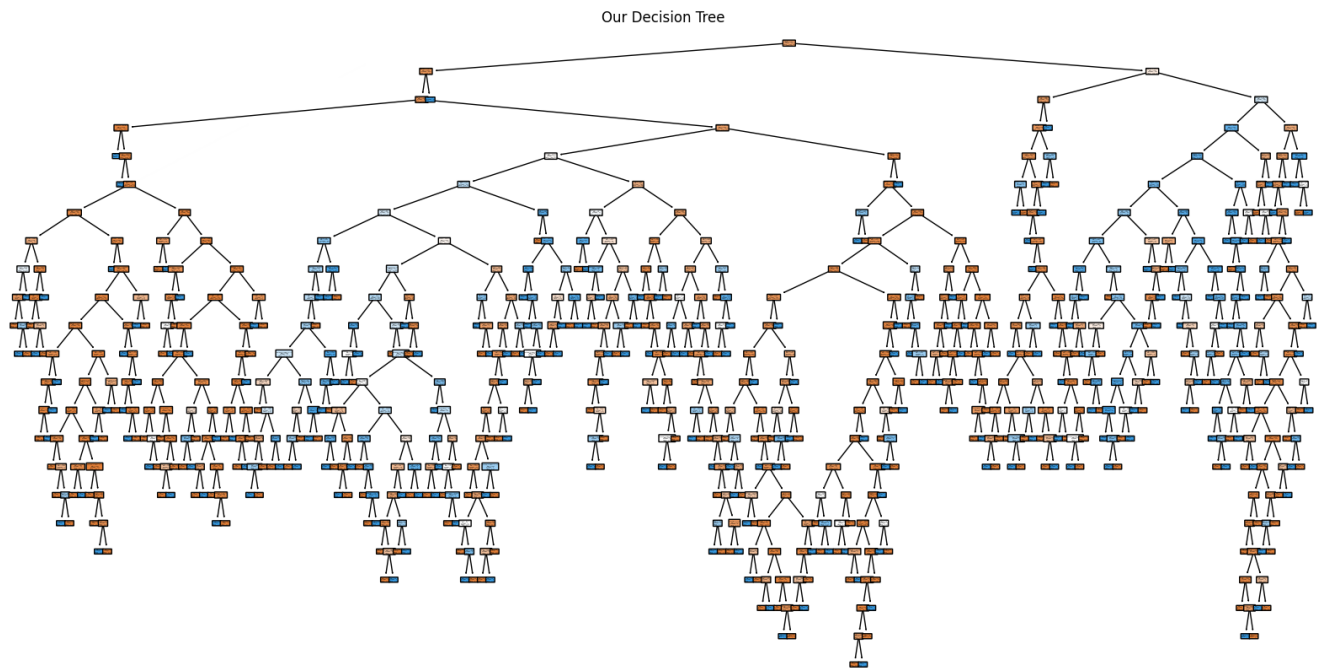
# Accessing the decision rules by examining the tree structure
print("Tree depth:", dt_default.get_depth())
print("Number of leaves:", dt_default.get_n_leaves())

# Predicted Probabilities and Classes
y_pred_default = dt_default.predict(X_test_encoded)
y_pred_proba_default = dt_default.predict_proba(X_test_encoded)[:, 1]
tree_pred_class = dt_default.predict(X_test_encoded)

# Plot Default Tree
plt.figure(figsize=(20, 10))
plot_tree(
    dt_default,
    feature_names=X_train_encoded.columns,
    class_names=['No', 'Yes'], # Assuming 0 = No, 1 = Yes
    filled=True,
    rounded=True
)
plt.title("Our Decision Tree")
plt.show()
print("Default Model Performance:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_default):.4f}")
print(f"ROC-AUC: {roc_auc_score(y_test, y_pred_proba_default):.4f}")
print(f"\nClassification Report:")
print(classification_report(y_test, y_pred_default))

print("\n5. Decision tree rules (first 20 rules):")
# Use the encoded feature names for the tree export
tree_rules = export_text(dt_default, feature_names=list(X_train_encoded.columns))
rules_lines = tree_rules.split('\n')
for i, rule in enumerate(rules_lines[:20]):
    if rule.strip():
        print(rule)
if len(rules_lines) > 20:
    print(f"... and {len(rules_lines) - 20} more rules")

```



4. Manipulate the default setting by changing the parameters, and try to improve the performance. Justify your choice.

We set max depth to 3 to make the model more interpretable and less prone to memorizing noise. We also required at least 2 samples in each leaf to reduce overfitting. Additionally, we set max features to 'none' instead of the default (sqrt) to improve predictive power. Finally, we are choosing information gain (entropy) instead of Gini impurity to possibly improve recall for the minority class.

Performance-wise, accuracy almost unchanged: $0.8686 \rightarrow 0.8651$ and ROC-AUC had a considerable improvement ($0.7738 \rightarrow 0.8636$), meaning the tuned tree is much better at ranking risky vs. safe cases.

The new tree is also much simple with only three levels of depth and therefore easier to apply to real-life business scenarios, where stakeholders can make assessments such as: "If debt-to-income is below ~35% and delinquencies are ≤ 1.5 , then the risk of default is low."

```

# Tuned Decision Tree
dt_tuned = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=3,
    min_samples_split=2,
    min_samples_leaf=2,
    max_features=None,
    random_state=42
)
dt_tuned.fit(X_train_encoded, y_train)

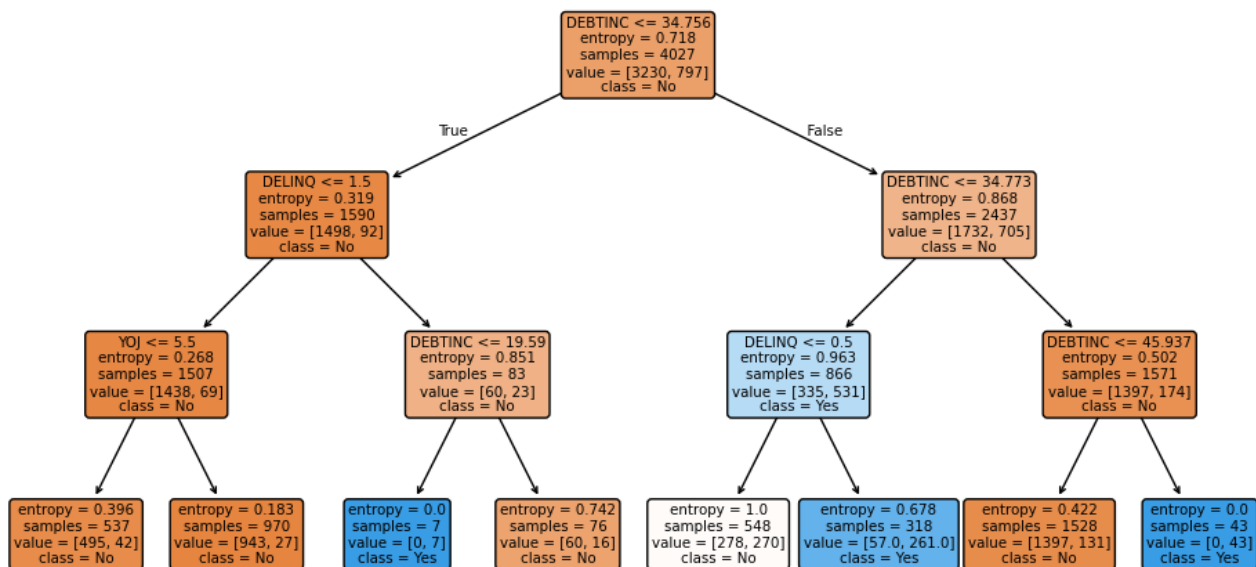
# Predictions
y_pred_tuned = dt_tuned.predict(X_test_encoded)
y_pred_proba_tuned = dt_tuned.predict_proba(X_test_encoded)[:, 1]

# Performance
print("Tuned Model Performance:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_tuned):.4f}")
print(f"ROC-AUC: {roc_auc_score(y_test, y_pred_proba_tuned):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_tuned))

# Plot tuned tree
plt.figure(figsize=(12, 6))
plot_tree(
    dt_tuned,
    feature_names=X_train_encoded.columns,
    class_names=['No', 'Yes'],
    filled=True,
    rounded=True
)
plt.title("Tuned Decision Tree (max_depth=3)")
plt.show()

```

Tuned Decision Tree (max_depth=3)



5. Once you nail down the model parameters, get the feature importance.

```
# Get default feature importances
importances = dt_default.feature_importances_
features = X_train_encoded.columns

feat_importance_df = pd.DataFrame({
    'Feature': features,
    'Default Importance': importances
}).sort_values(by='Default Importance', ascending=False)

print(feat_importance_df)
```

	Feature	Default Importance
9	DEBTINC	0.344999
5	DELINQ	0.121997
6	CLAGE	0.085850
0	LOAN	0.079011
8	CLNO	0.077751
1	MORTDUE	0.069487
3	YOJ	0.061025
2	VALUE	0.058084
7	NINQ	0.035910
4	DEROG	0.029491
10	REASON_HomeImp	0.008576
15	JOB_Self	0.007749
11	JOB_Office	0.006537
13	JOB_ProfExe	0.006225
14	JOB_Sales	0.004245
12	JOB_Other	0.003063

```
# Get tuned feature importances
importances = dt_tuned.feature_importances_
features = X_train_encoded.columns

tunedfeat_importance_df = pd.DataFrame({
    'Feature': features,
    'Tuned Importance': importances
}).sort_values(by='Tuned Importance', ascending=False)

print(tunedfeat_importance_df)
```

	Feature	Tuned Importance
9	DEBTINC	0.887947
5	DELINQ	0.098791
3	YOJ	0.013262
1	MORTDUE	0.000000
2	VALUE	0.000000
4	DEROG	0.000000
6	CLAGE	0.000000
0	LOAN	0.000000
7	NINQ	0.000000
8	CLNO	0.000000
10	REASON_HomeImp	0.000000
11	JOB_Office	0.000000
12	JOB_Other	0.000000
13	JOB_ProfExe	0.000000
14	JOB_Sales	0.000000
15	JOB_Self	0.000000

Problem 5. Please do not use Python to answer this question. Consider the following data set that contains some hypothetical information about 10 USF students who have taken the machine learning class. We want to construct a decision tree to predict who passes the final exam.

Instances	Department	Master?	Number of courses taken	Pass?
1	ISM	Y	3	Y
2	ISM	Y	7	N
3	ISM	N	4	Y
4	IE	N	4	N
5	IE	Y	7	N
6	ISM	Y	4	Y
7	IE	N	5	Y
8	ISM	N	6	Y
9	IE	Y	1	Y
10	ISM	Y	5	N

- (a) Given the 10 instances in the table above, calculate the information gain of each candidate split point. Which attribute do you choose at the root of your tree?

Initial Entropy: $-(6/10)\log_2(6/10) - (4/10)\log_2(4/10) = 0.971$

Department:

ISM: $-(4/6)\log_2(4/6) - (2/6)\log_2(2/6) = 0.918$

IE: $-(2/4)\log_2(2/4) - (2/4)\log_2(2/4) = 1$

Department: $0.971 - [(6/10)*0.918 + (4/10)*1] = 0.020$

Master?

Yes: 3Y and 3N, therefore = 1

No: $-(3/4)\log_2(3/4) - (1/4)\log_2(1/4) = 0.811$

Master: $0.971 - [(6/10)*1 + (4/10)*0.811] = 0.047$

Number of Courses (taking the median as the starting point):

<= 4.5: (4Ys, 1Ns)

$-(4/5)\log_2(4/5) - (1/5)\log_2(1/5) = 0.722$

>4.5: (2Ys, 3Ns)

$-(2/5)\log_2(2/5) - (3/5)\log_2(3/5) = 0.971$

Number of Courses: $0.971 - [(5/10)*0.722 + (5/10)*0.971] = 0.124$

Noting that there are 2 instances of [Pass] = "N" for [Number of Courses Taken] = 7 out of 10 entries, let's study cut point value 6:

<= 6:

$-(6/8)\log_2(6/8) - (2/8)\log_2(2/8) = 0.811$

>6:

Pure Node = 0

Number of Courses: $0.971 - [(8/10)*0.811 + (2/10)*0] = 0.322$

As [Number of Courses] demonstrated the highest information gain at cut point 6 in comparison to the other variables, it is chosen as our tree root.

- (b) Construct the entire tree to its full depth. Show your computations and the final tree.
Before proceeding with the creation of the tree, let's first calculate the information gain for the second level to identify our next split.

New subset parent entropy (6 Ys and 2 Ns left):

$$-(6/8)\log_2(6/8) - (2/8)\log_2(2/8) = \mathbf{0.811}$$

Department:

$$\mathbf{ISM:} -(4/5)\log_2(4/5) - (1/5)\log_2(1/5) = 0.722$$

$$\mathbf{IE:} -(2/3)\log_2(2/3) - (1/3)\log_2(1/3) = 0.918$$

$$\mathbf{Department:} 0.811 - [(5/8)* 0.722 + (3/8)* 0.918] = \mathbf{0.0157}$$

Master?

$$\mathbf{Yes:} -(3/4)\log_2(3/4) - (1/4)\log_2(1/4) = 0.811$$

$$\mathbf{No:} -(3/4)\log_2(3/4) - (1/4)\log_2(1/4) = 0.811$$

$$\mathbf{Master:} 0.811 - [(4/8)*0.811 + (4/8)*0.811] = \mathbf{0}$$

Number of Courses (testing for 3.5 this time):

<= 3.5:

Pure Node = 0

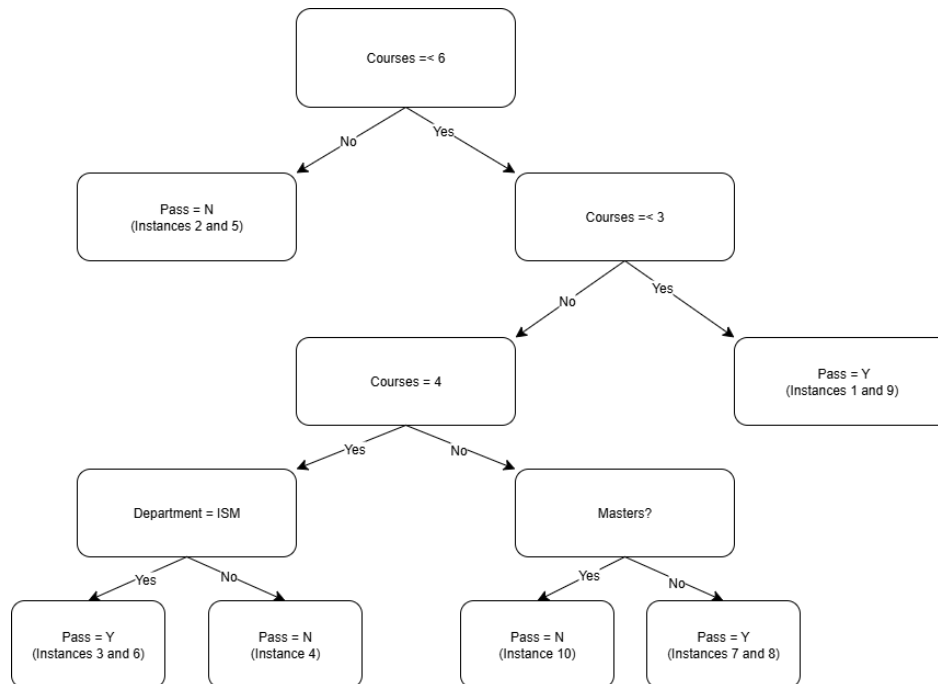
> 3.5:

$$-(4/6)\log_2(4/6) - (2/6)\log_2(2/6) = 0.918$$

$$\mathbf{Number\ of\ Courses:} 0.811 - [(2/8)*0 + (6/8)*0.918] = \mathbf{0.123}$$

With that, we select again [Number of Classes] for our split rule, now at cut point 3.5.

With more computations, a decision tree example would be the following:



(c) Using your decision tree, provide two decision rules that the instructor can use to predict if a student is going to pass or fail. Justify your choice.

1. If [Number of Courses] ≤ 6, THEN: Pass = N. (Student Fails)
2. If [Number of Courses] = 4 AND [Department] = ISM, THEN: Pass = Y. (Student Passes)

The rules above were selected because, for the small dataset, they would be understood as perfect rules with perfect predictions. Nevertheless, with a more extensive dataset, these rules could very likely be translated into an overfitting attempt.

- (d) If we use Gini index to construct a decision tree, which variable will be the root node? Explain.

6Ys and 4Ns

$$\text{Gini} = 1 - (0.6^2 + 0.4^2) = 0.48$$

Department:

ISM: $1 - (4/6)^2 - (2/6)^2 = 0.444$

IE: 2Ys and 2Ns, therefore = 0.5

Department: $0.48 - [0.6 * 0.444 + 0.4 * 0.5] = 0.0133$

Master?

Yes: 3Y and 3N, therefore = 0.5

No: $1 - (3/4)^2 - (1/4)^2 = 0.375$

Master: $0.48 - [0.6 * 0.5 + 0.4 * 0.375] = 0.03$

Number of Courses (taking the median as the starting point):

<= 4.5: (4Ys, 1Ns)

$$1 - (4/5)^2 - (1/5)^2 = 0.32$$

>4.5: (2Ys, 3Ns)

$$1 - (2/5)^2 - (3/5)^2 = 0.48$$

Number of Courses: $0.48 - [0.5 * 0.32 + 0.5 * 0.48] = 0.08$

Given the fact that [Number of Courses] displays the highest information gain after the Gini index computations, it would be the variable selected as our root – just like via entropy.